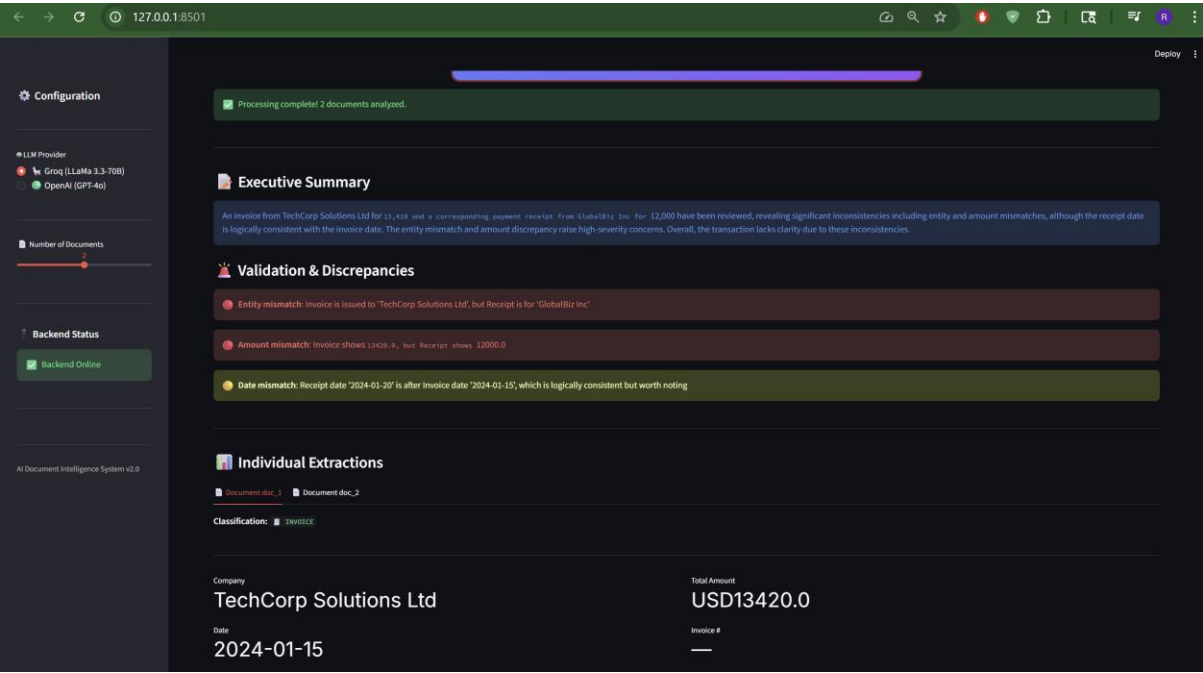
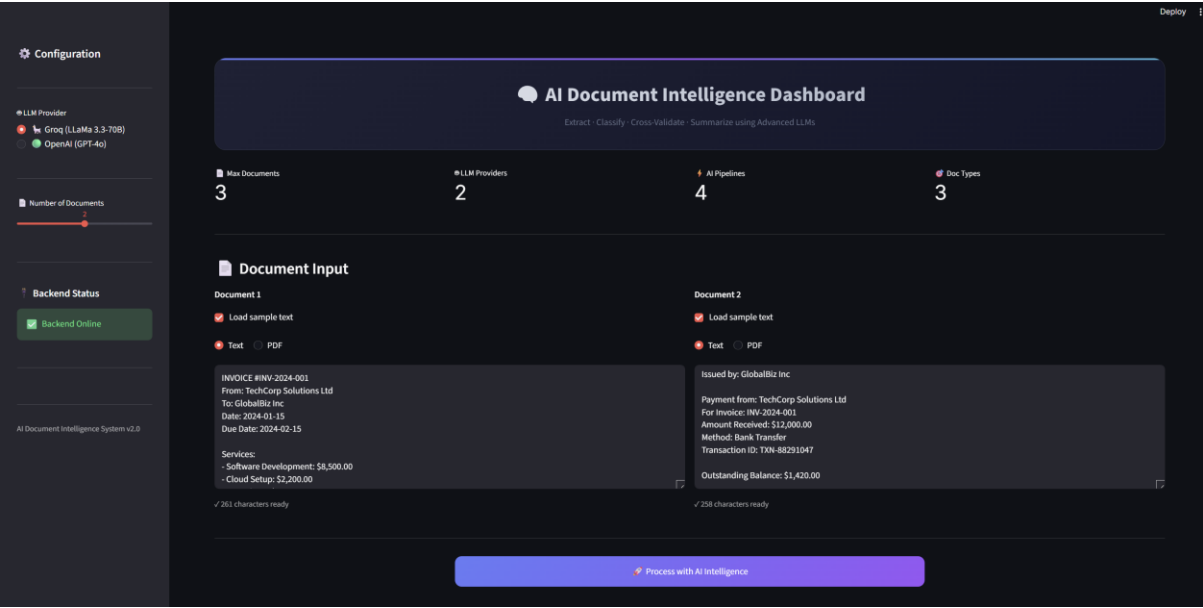


AI Document Intelligence System (Multi-Doc Extraction + Validation + Reasoning)



Configuration

LLM Provider

Groq (LLaMa 3.3-70B)

OpenAI (GPT-4o)

Number of Documents

2

Backend Status

Backend Online

AI Document Intelligence System v2.0

Individual Extractions

Document doc_1

Document doc_2

Classification: INVOICE

Company

TechCorp Solutions Ltd

Total Amount

USD13420.0

Date

2024-01-15

Invoice #

Full Extracted Data

```
{
  "company_name": {
    "value": "TechCorp Solutions Ltd"
    "confidence": 1
  },
  "date": {
    "value": "2024-01-15"
    "confidence": 1
  },
  "total_amount": {
    "value": 13420
    "confidence": 1
  },
  "currency": {
    "value": "USD"
    "confidence": 0.8
  }
}
```

Configuration

LLM Provider

Groq (LLaMa 3.3-70B)

OpenAI (GPT-4o)

Number of Documents

1

Backend Status

Backend Online

AI Document Intelligence System v2.0

Individual Extractions

Document doc_1

Document doc_2

Classification: RECEIPT

Company

GlobalBiz Inc

Total Amount

USD12000.0

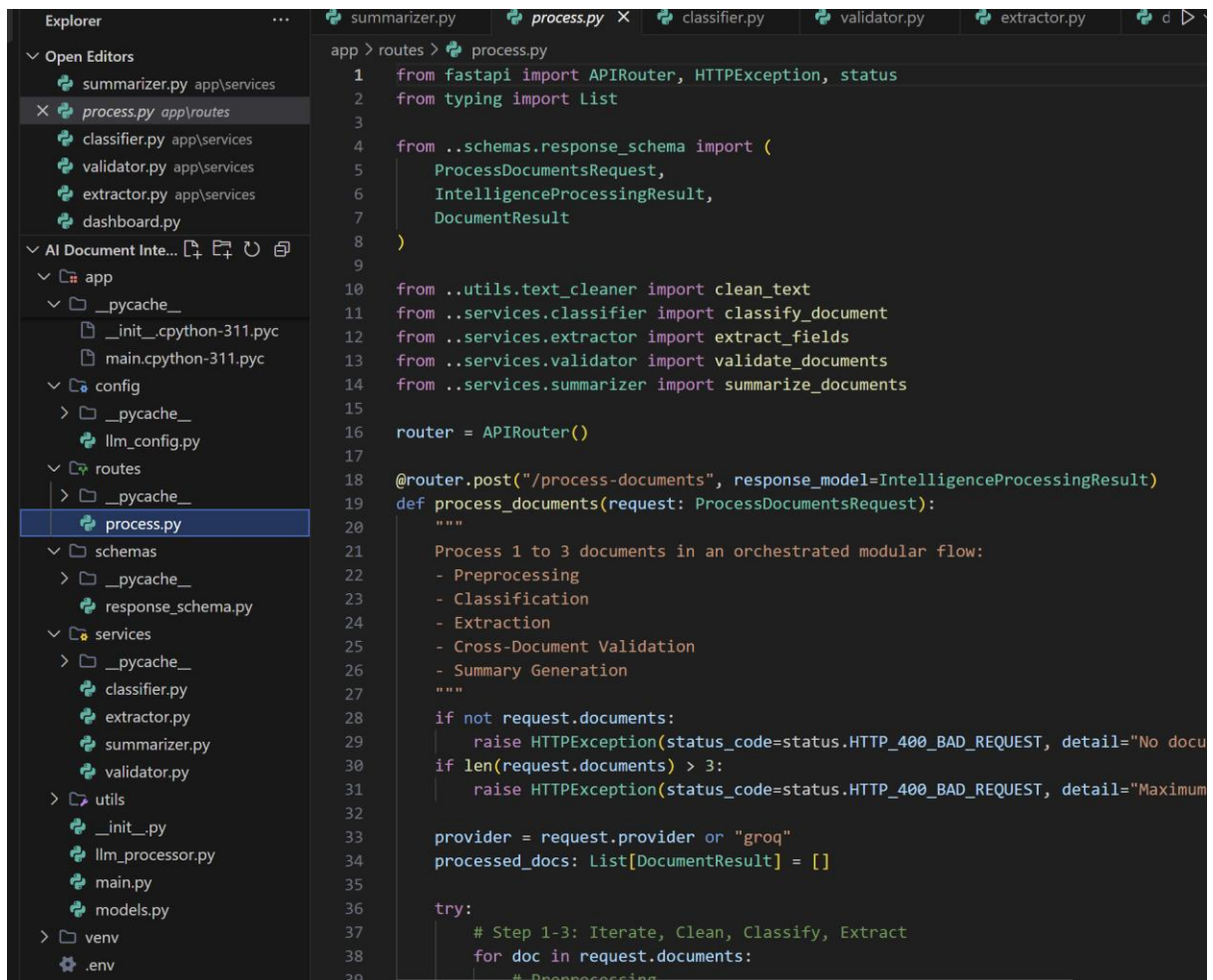
Date

2024-01-20

Invoice #

Full Extracted Data

```
{
  "company_name": {
    "value": "GlobalBiz Inc"
    "confidence": 1
  },
  "date": {
    "value": "2024-01-20"
    "confidence": 1
  },
  "total_amount": {
    "value": 12000
    "confidence": 1
  },
  "currency": {
    "value": "USD"
    "confidence": 0.8
  }
}
```



The image shows a code editor interface with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'app', 'schemas', 'services', and 'utils'. The code editor shows the 'process.py' file in the 'routes' folder, containing FastAPI route definitions and logic for processing documents.

```
1 from fastapi import APIRouter, HTTPException, status
2 from typing import List
3
4 from ..schemas.response_schema import (
5     ProcessDocumentsRequest,
6     IntelligenceProcessingResult,
7     DocumentResult
8 )
9
10 from ..utils.text_cleaner import clean_text
11 from ..services.classifier import classify_document
12 from ..services.extractor import extract_fields
13 from ..services.validator import validate_documents
14 from ..services.summarizer import summarize_documents
15
16 router = APIRouter()
17
18 @router.post("/process-documents", response_model=IntelligenceProcessingResult)
19 def process_documents(request: ProcessDocumentsRequest):
20     """
21     Process 1 to 3 documents in an orchestrated modular flow:
22     - Preprocessing
23     - Classification
24     - Extraction
25     - Cross-Document Validation
26     - Summary Generation
27     """
28     if not request.documents:
29         raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="No documents provided")
30     if len(request.documents) > 3:
31         raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Maximum 3 documents allowed")
32
33     provider = request.provider or "groq"
34     processed_docs: List[DocumentResult] = []
35
36     try:
37         # Step 1-3: Iterate, Clean, Classify, Extract
38         for doc in request.documents:
39             # Preprocessing
```